

DOCUMENTACIÓN DE PRUEBAS

CONSULTORIO JURÍDICO

1. INTRODUCCIÓN

Las pruebas son una parte fundamental del desarrollo de la aplicación Django “Consultorio Jurídico”. Este documento describe la estructura, organización y ejecución de las pruebas unitarias implementadas, así como el estado actual de cobertura y las mejoras planificadas.

Objetivos de las pruebas:

- Validar el correcto funcionamiento de los modelos y lógica de negocio
- Detectar errores de forma temprana
- Garantizar la integridad de los datos
- Facilitar el mantenimiento del sistema

2. ESTRUCTURA DE PRUEBAS

Ubicación de archivos:

Ruta	Descripción
usuarios/tests.py	Pruebas del módulo de usuarios
citas/tests.py	Pruebas del módulo de citas
consentimiento/tests.py	Sin pruebas actualmente
reportes/tests.py	Sin pruebas actualmente
tasks/tests.py	Sin pruebas actualmente
manage.py	Script de gestión del proyecto Django

3. MÓDULOS PROBADOS

3.1 Módulo: Usuarios

Archivo: usuarios/tests.py

Modelos probados:

- Usuario (heredado de AbstractUser)

Funcionalidades cubiertas:

Funcionalidad	Estado
Creación de usuarios	OK
Rol por defecto	OK
Autenticación por documento	OK
Eliminación de campo username	OK
Representación en string	OK
Validación de campos únicos	OK
Manejo de login/logout	OK
Validación de formularios	OK

Clases de prueba:

Clase	Descripción
UsuarioModelTest	Validación del modelo Usuario
AutenticacionPorDocumentoTest	Pruebas de autenticación
LoginFormTest	Validación de formularios

3.2 Módulo: Citas

Archivo: citas/tests.py

Modelos probados:

Modelo	Descripción
HorarioDisponible	Gestión de horarios
Caso	Gestión de casos jurídicos
Cita	Gestión del ciclo de citas
RegistroAsistencia	Registro de asistencia

Funciones auxiliares:

Función	Descripción
crear_usuario	Crea usuario de prueba
crear_horario	Crea horario disponible
crear_caso	Crea caso jurídico
crear_cita	Crea cita

Resumen de clases de prueba:

Clase	Propósito
HorarioDisponibleModelTest	Validación de horarios
CasoModelTest	Validación de casos
CitaModelTest	Máquina de estados de citas
RegistroAsistenciaModelTest	Validación de asistencia

Máquina de estados de citas:

Estado actual	Transición permitida
Pendiente	Confirmada, Cancelada
Confirmada	Cumplida, Cancelada
Cumplida	No permite cambios

4. EJECUCIÓN DE PRUEBAS

Requisitos:

Requisito	Descripción
Dependencias	Instalar desde requirements.txt
Django	Versión 6.0.3

Comandos principales:

Acción	Comando
Ejecutar todas las pruebas	<code>python manage.py test</code>
Ejecutar módulo usuarios	<code>python manage.py test usuarios</code>
Ejecutar módulo citas	<code>python manage.py test citas</code>
Ejecutar una clase específica	<code>python manage.py test usuarios.tests.UsuarioModelTest</code>
Ejecutar una prueba específica	<code>python manage.py test usuarios.tests.UsuarioModelTest.test_usuario_creado_correctamente</code>

5. PRUEBAS POR MÓDULO

Resumen:

Módulo	Tipo de prueba	Cantidad	Estado
Usuarios	Modelo	9	Activas
Usuarios	Autenticación	5	Activas
Usuarios	Formularios	3	Activas
Citas	Horarios	3	Activas
Citas	Casos	3	Activas
Citas	Citas	12	Activas
Citas	Asistencia	3	Activas

6. COBERTURA ACTUAL

Resumen general:

Módulo	Modelos	Pruebas	Estado
usuarios	1	17	Bien cubierto
citas	4	21	Bien cubierto
consentimiento	1	0	Sin pruebas
reportes	1	0	Sin pruebas
tasks	1	0	Sin pruebas

Indicadores:

Métrica

Valor

Total de pruebas	38
Módulos cubiertos	2/5
Cobertura estimada	40%

Áreas cubiertas:

- Autenticación de usuarios
- Validación de campos únicos
- Ciclo de vida de citas
- Máquina de estados
- Integridad referencial

Áreas sin cobertura:

- Vistas (views)
- APIs / serializers
- Validadores personalizados
- Reportes
- Consentimiento
- Tareas